



NORTHERN UNIVERSITY

OF BUSINESS & TECHNOLOGY KHULNA

Department of Computer Science and Engineering

[Software Requirement Specification (SRS)]

Project Title: AI-Based Smart Quiz System.

Course Code: CSE 4204

Course Title: Mobile Computing Lab

Submitted By:

Team Name : CSE4204-8A-T03

Section : 8A

Team Members IDs: i) Name: Chaon Das

ID :11210320629

ii) Name: Rakibur Rahman Mishan

ID :11220320829

iii) Name: Sudipto Das [Team Leader]

ID :11220320830

iv) Name: Aditi Roy

ID :11220320846

GitHub Repository: <https://github.com/sudipto224/AI-based-Smart-Quiz-System>

Submitted to:

Md. Riaz Mahmud

Designation: Assistant Professor

Department of CSE

Date of Submission: 08/06/2026

1. Introduction

1.1 Project Title

AI-Based Smart Quiz System

1.2 Project Overview

The AI-Based Smart Quiz System is a web-based platform built using Laravel 12, MySQL, and HTML/CSS/JavaScript. It supports two user roles Teacher and Student and provides a fully featured quiz environment. Teachers can create and manage courses, add multiple-choice questions (MCQs), and leverage Google's Gemini AI API to automatically generate quiz questions for the Mobile Computing Lab course. Students can log in, take timed quizzes, and receive instant feedback with correct answers and explanations. The system also includes a cheating detection module that monitors suspicious quiz behavior for the Mobile Computing Lab course, and a leaderboard that ranks students by score and completion time.

1.3 Problem Statement

Traditional quiz systems lack intelligent automation and integrity monitoring. Teachers spend significant time manually crafting quiz questions, and there is no reliable mechanism to detect dishonest behavior during online assessments. Students receive limited feedback after attempting quizzes, reducing the educational value of assessments. Furthermore, ranking systems often fail to differentiate between equally scoring students fairly. This project addresses these gaps by integrating AI-powered question generation, automated cheating detection, detailed result feedback, and a time-aware leaderboard into a single, unified quiz platform.

1.4 Objectives

- To provide an intuitive platform for teachers to create and manage courses and MCQ-based quizzes.
- To automate quiz question generation using the Google Gemini API for the Mobile Computing Lab course.
- To deliver a timed quiz experience for students with per-question countdown timers.
- To provide students with detailed post-quiz feedback including correct answers and explanations.

- To detect and flag potentially dishonest quiz attempts through tab-switch monitoring and response-time analysis for the Mobile Computing Lab course.
- To display a fair, score-and-time-based leaderboard for each course.

1.5 Scope of the Project

Included in this project:

- User registration and login for Teacher and Student roles.
- Teacher panel for creating and managing courses and MCQ questions.
- AI-powered question generation via Google Gemini API (restricted to the Mobile Computing Lab course).
- Student panel for browsing courses and taking timed quizzes.
- Automated cheating detection using tab-switch tracking and average time analysis (restricted to the Mobile Computing Lab course).
- Post-quiz result display with explanations and correct answers.
- Course-wise leaderboard ranked by score and completion time.
- Teacher view for suspicious attempt reports for the Mobile Computing Lab course.

Excluded from this project:

- Mobile native applications (iOS/Android).
- Payment or subscription management features.
- Real-time collaborative or multi-teacher course editing.
- Support for question types other than multiple-choice (MCQ).
- Video or multimedia content within quiz questions.
- AI question generation for courses other than Mobile Computing Lab.

2. Functional Requirements

2.1 User Registration & Authentication

FR-01: The system shall allow new users to register by providing their full name, email address, password, and selecting a role (Teacher or Student).

FR-02: The system shall validate that the email address is unique and in a valid format during registration.

FR-03: The system shall securely hash user passwords before storing them in the database.

FR-04: Registered users shall be able to log in using their email and password.

FR-05: Upon successful login, the system shall redirect users to their role-specific dashboard (Teacher Panel or Student Dashboard).

FR-06: The system shall provide a logout function that terminates the user session.

2.2 Teacher Panel – Course Management

FR-07: Teachers shall be able to create a new course by entering a course name, a description, and a configurable time-per-question value (in seconds; default is 30 seconds).

FR-08: Teachers shall be able to view a list of all courses they have created.

FR-09: Teachers shall be able to edit the name, description, and time-per-question of any course they own.

FR-10: Teachers shall be able to delete a course, which also removes all associated questions and quiz attempts.

2.3 Teacher Panel – Question Management (Manual)

FR-11: For each course, teachers shall be able to add MCQ questions containing: question text, four answer options (A, B, C, D), the correct answer (one of the four options), and an explanation.

FR-12: Teachers shall be able to view the full list of questions associated with a specific course.

FR-13: Teachers shall be able to edit any field of an existing question (text, options, correct answer, explanation).

FR-14: Teachers shall be able to delete individual questions from a course.

2.4 AI Feature 1 – Automated Question Generation (Gemini API)

Note: This feature is exclusively available for the course named "Mobile Computing Lab".

FR-15: When a teacher adds or edits a question for the Mobile Computing Lab course, the question form shall display an additional section titled "AI Question Generator".

FR-16: The AI Question Generator section shall contain a text input field for a topic (e.g., "Mobile IP") and a button labeled "Generate with AI".

FR-17: When the teacher clicks "Generate with AI", the system shall send a structured prompt to the Google Gemini API (free tier) requesting one MCQ on the specified topic.

FR-18: The system shall expect the Gemini API to return a JSON object containing: the question text, four answer options, the correct answer (as an exact option string), and an explanation.

FR-19: The system shall parse the returned JSON and automatically populate all fields in the question form (question text, options A–D, correct answer, explanation).

FR-20: The teacher shall be able to review, edit, and save or discard the AI-generated question.

FR-21: If the Gemini API call fails or returns malformed data, the system shall display a user-friendly error message and leave the form fields unchanged.

2.5 Student Panel – Taking Quizzes

FR-22: Students shall be able to view a list of all available courses on their dashboard.

FR-23: Students shall be able to start a quiz for any course by clicking a "Start Quiz" button on the course detail page.

FR-24: The quiz shall present one MCQ at a time, with a countdown timer based on the course's configured time-per-question value.

FR-25: When the countdown reaches zero, the system shall automatically record the current question (with no answer selected if none was chosen) and advance to the next question.

FR-26: The student shall be able to select one of four answer options per question.

FR-27: After the final question, the system shall automatically submit the quiz and record the attempt.

2.6 AI Feature 2 – Cheating Detection (Tab Switch & Fast Answers)

Note: This feature is exclusively active during quizzes for the Mobile Computing Lab course.

FR-28: During a Mobile Computing Lab quiz, the system shall use the JavaScript visibilitychange event to detect when the student switches to another browser tab or window.

FR-29: The system shall maintain a running count (tab_switch_count) of the number of times the tab visibility changes to hidden during the quiz session.

FR-30: The system shall record the quiz start timestamp (in milliseconds) when the quiz begins.

FR-31: Upon quiz submission, the system shall calculate the total time taken (in seconds) and the average time per question (total seconds / number of questions).

FR-32: An attempt shall be flagged as suspicious (`is_suspicious = true`) if either of the following conditions is met: (a) `tab_switch_count` is greater than 0, or (b) average time per question is less than 5 seconds.

FR-33: The system shall store the following per attempt in the database: `tab_switch_count`, `average_time_per_question`, and `is_suspicious` (boolean).

2.7 Result & Explanation Display

FR-34: Immediately after quiz submission, the student shall see a result summary page displaying their score (e.g., 7/10).

FR-35: The result page shall display, for each question: the question text, the student's chosen answer, the correct answer, and the explanation.

FR-36: Students shall be able to retake a quiz; each attempt is stored separately.

2.8 Leaderboard

FR-37: The system shall display a leaderboard for each course showing student rankings.

FR-38: Rankings shall be determined by highest score first. In the case of a tie in scores, the student with the shorter total completion time shall rank higher.

FR-39: Only the best attempt (highest score; if still tied, shortest time) per student shall appear on the leaderboard.

2.9 Teacher Panel – View Suspicious Attempts

FR-40: Teachers shall have access to a "Suspicious Attempts" report page for the Mobile Computing Lab course.

FR-41: The report shall list all quiz attempts flagged as suspicious, displaying: student name, score, `tab_switch_count`, `average_time_per_question`, and the date and time of the attempt.

FR-42: The report shall only be visible to teachers and shall not be accessible to students.

3. Non-Functional Requirements

3.1 Performance

NFR-01: All standard page loads (dashboards, course list, leaderboard) shall render within 3 seconds under normal network conditions.

NFR-02: The Gemini API call for AI question generation shall complete and display results to the teacher within 10 seconds. A loading indicator shall be shown during the request.

NFR-03: Quiz timer accuracy shall be maintained within ± 1 second across all supported browsers.

3.2 Security

NFR-04: All user passwords shall be hashed using a strong, industry-standard algorithm (e.g., bcrypt) before being stored in the database.

NFR-05: The Google Gemini API key shall be stored as a server-side environment variable and shall never be exposed in client-side code or version control.

NFR-06: All authenticated routes shall be protected by session-based middleware; unauthorized access attempts shall be redirected to the login page.

NFR-07: The application shall protect against common web vulnerabilities, including SQL injection (via ORM parameterized queries), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

3.3 Usability

NFR-08: The system shall provide a responsive user interface compatible with modern desktop and mobile browsers.

NFR-09: Navigation between key sections (courses, quiz, results, leaderboard) shall require no more than two clicks from the main dashboard.

NFR-10: Error messages (e.g., failed login, AI generation error) shall be clear, concise, and non-technical.

3.4 Reliability

NFR-11: If the Gemini API is unavailable or returns an unexpected response, the system shall display a clear fallback error message and allow the teacher to proceed with manual question entry without loss of any entered data.

NFR-12: Quiz attempt data (including tab_switch_count and timestamps) shall be stored only upon successful submission; partially abandoned quizzes shall not corrupt the database.

3.5 Maintainability

NFR-13: The system shall follow the MVC (Model-View-Controller) architectural pattern as enforced by the Laravel framework.

NFR-14: All business logic related to cheating detection and AI question generation shall be encapsulated in dedicated service classes or controllers to facilitate future modifications.

NFR-15: The codebase shall use database migrations for schema management, enabling reproducible environment setup.

3.6 Scalability

NFR-16: The database schema shall support an unrestricted number of courses, questions, students, and quiz attempts without architectural changes.

NFR-17: The system shall be designed so that AI question generation can be extended to additional courses in the future by modifying a configuration value, without requiring significant code restructuring.

4. User Roles

4.1 Teacher

The Teacher is an authenticated user responsible for content creation and monitoring.

Teacher responsibilities include:

- Registering for an account with the Teacher role.
- Creating, updating, and deleting courses with configurable quiz timings.
- Adding, editing, and deleting MCQ questions for their courses.
- Using the AI Question Generator to automatically generate questions for the Mobile Computing Lab course.
- Reviewing the Suspicious Attempts report to monitor academic integrity for the Mobile Computing Lab course.
- Viewing the course leaderboard to monitor student performance.

4.2 Student

The student is an authenticated user who consumes course content through quizzes. Student responsibilities include:

- Registering for an account with the Student role.
- Browsing available courses on the student dashboard.
- Taking timed quizzes for any available course.
- Reviewing quiz results, correct answers, and explanations after each attempt.

- Viewing the course leaderboard to check personal ranking.

5. Use Cases

The following table summarizes the primary use cases of the AI-Based Smart Quiz System, describing the interactions between system actors and the platform.

Use Case	Actor	Action	System Response
Register Account	Teacher / Student	Fills registration form with name, email, password, and selects role.	System validates input, hashes password, stores user record, and redirects to the appropriate dashboard.
User Login	Teacher / Student	Enters registered email and password on the login page.	System validates credentials and redirects the user to the Teacher Panel or Student Dashboard based on role.
Create a Course	Teacher	Enters course name, description, and time-per-question (seconds), then clicks 'Create'.	System stores the course in the database and displays it in the Teacher's course list.
Add a Question Manually	Teacher	Navigates to a course, enters question text, four options, correct answer, explanation, and clicks 'Save'.	System validates the question data and stores the MCQ in the database linked to that course.
Generate Question with AI	Teacher	On the Mobile Computing Lab question form, enters a topic (e.g., 'Mobile IP') and clicks 'Generate with AI'.	System calls the Gemini API, receives a JSON MCQ response, parses it, and auto-fills the question form fields.
Take a Quiz	Student	Selects a course and clicks 'Start Quiz'; answers questions within the countdown timer.	System presents one question at a time with a countdown. If time expires, the system auto-advances to the next question.

Use Case	Actor	Action	System Response
Submit Quiz and View Result	Student	Answers all questions and clicks 'Submit' (or auto-submits on final timeout).	System calculates the score, stores the attempt, and displays score, chosen answers, correct answers, and explanations.
Cheating Detection Triggers	System (automated)	Monitors tab switches (visibilitychange) and calculates average time per question upon submission.	If <code>tab_switch_count > 0</code> or average time < 5 seconds, the attempt is flagged as suspicious and stored with <code>is_suspicious = true</code> .
View Suspicious Attempts	Teacher	Navigates to the 'Suspicious Attempts' section for the Mobile Computing Lab course.	System displays a list of flagged attempts with student name, score, tab switch count, average time, and attempt date.
View Leaderboard	Student / Teacher	Clicks 'Leaderboard' for a specific course.	System displays rankings by highest score first; ties are broken by shortest completion time. Only the best attempt per student is shown.